

AlphaClifford: Efficient Clifford Synthesis and Transpilation with Model-based RL

Daniele Lizzio Bosco

University of Udine

Udine, Italy

University of Naples “Federico II”
Naples, Italy

lizziosbosco.daniele@spes.uniud.it

Jacopo Cossio

University of Udine

Udine, Italy

cossio.jacopo@spes.uniud.it

Carla Piazza

University of Udine

Udine, Italy

carla.piazza@uniud.it

Giuseppe Serra

University of Udine

Udine, Italy

giuseppe.serra@uniud.it

Abstract—Clifford circuits play a foundational role in quantum computing, particularly due to their importance in quantum error correction and fault-tolerant logical synthesis. While these circuits can be efficiently simulated and represented as symplectic matrices, standard synthesis methods—such as the Aaronson-Gottesman algorithm—often yield sub-optimal circuits with excessively high gate counts. In this work, we introduce AlphaClifford, a model-based Reinforcement Learning framework powered by Monte Carlo Tree Search, designed to efficiently synthesize Clifford circuits from the fundamental gate set composed of H , S , and $CNOT$. By modeling the state space through the algebraic properties of the symplectic group, AlphaClifford effectively explores this combinatorial space to minimize overall circuit cost. For unconstrained Clifford optimization, our approach achieves a consistent reduction in both total and two-qubit ($CNOT$) gate counts compared to state-of-the-art synthesis heuristics, despite operating with a strictly less expressive gate set. Furthermore, we demonstrate the broad applicability of our framework on two additional tasks: hardware-constrained Clifford transpilation, where we outperform existing RL-based compilers, and as a post-synthesis optimization component within a full Clifford+ T logical synthesis pipeline. Our results underscore that model-based RL is highly effective at addressing the combinatorial complexities of quantum compilation, offering a scalable pathway to mitigate hardware constraints in both near-term and future fault-tolerant quantum devices.

Index Terms—Reinforcement Learning, Clifford Circuits, Quantum Synthesis, Quantum Optimization, Circuit Compilation

I. INTRODUCTION

The field of quantum computing is rapidly approaching the era of “quantum utility” [1], marking a pivotal transition where quantum devices cease to be experimental research instruments and begin to execute practical, large-scale applications. These applications are expected to span both scientific domains, such as Quantum Chemistry [2] and High Energy Physics [3], and industrial use cases, including discrete optimization [4] and finance [5].

Despite significant recent advancements in quantum hardware capabilities—most notably in error rate reduction and

coherence time improvements [6], [7]—the execution of quantum algorithms remains fundamentally constrained by circuit depth and the total number of physical gates. To mitigate these hardware limitations, quantum circuit optimization plays a critical role. As equivalent unitary transformations can be realized by different sequences of quantum gates, the objective of quantum circuit synthesis [8]–[12] is to find a sequence of gates that implements a target circuit (or its mathematical representation) while minimizing a specific cost metric, such as overall circuit depth or the number of two-qubit entangling gates.

A closely related challenge is quantum circuit transpilation [13], [14], which involves mapping a logical circuit onto a specific physical architecture. This requires decomposing the target circuit into a native gate set and respecting strict hardware constraints, such as the limited connectivity map of the physical qubits [15].

The problems of quantum circuit synthesis and transpilation have been extensively explored in the literature, utilizing a wide array of heuristic, algebraic, and search-based techniques [9], [16]–[18]. Recently, Reinforcement Learning (RL) has emerged as a highly effective paradigm for tackling these computationally hard optimization problems [19]–[27]. By learning complex patterns rather than relying on heuristic rules, RL-based approaches have demonstrated the ability to discover novel, non-intuitive gate sequences that rule-based compilers might otherwise miss.

In this work, we propose a RL-based technique to address both the synthesis and transpilation tasks specifically for the set of Clifford circuits. Generated strictly by the Hadamard (H), Phase (S), and Controlled-NOT ($CNOT$) gates, Clifford circuits are one of the most studied gate sets, and play a central role in the logical synthesis pipeline, particularly within the context of Clifford+ T decomposition [21].

A key advantage of Clifford gates is that they admit an efficient classical representation via symplectic matrices [28]. We exploit this compact representation to develop a RL framework designed to learn highly efficient syntheses and transpilation strategies for Clifford circuits.

Inspired by recent successes on similar tasks from AlphaTensor [19] and AlphaCNOT [27], our method formulates

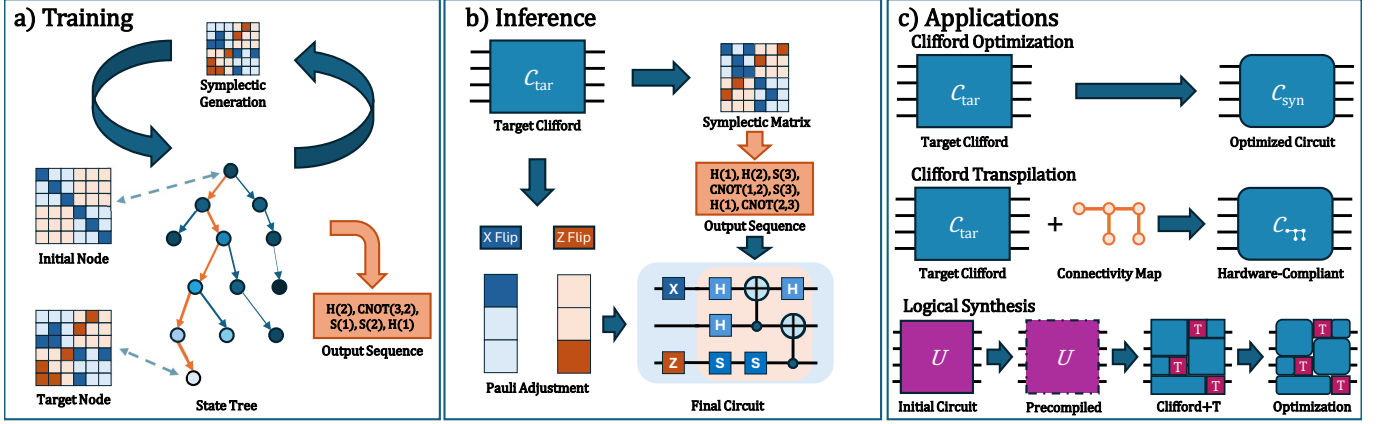


Fig. 1: **a)** We train our model to reproduce a target symplectic matrix. Each symplectic matrix is modelled as the node of a tree, where nodes are connected if the corresponding matrices can be reached with an operation from H , S , and $CNOT$. We train a MCTS-based model as a pair of value network (for nodes, e.g. lighter blue corresponds to a higher value) and policy network (for edges, e.g. wider arrows corresponds to higher probabilities). **b)** Given a target Clifford circuit C_{tar} , we utilize our model to synthesize a circuit C with the same symplectic matrix, i.e. $P \cdot C \equiv C_{tar}$ for a suitable Pauli P . We then compute the P from initial circuit, and we apply the corresponding gates to the obtained circuit. **c)** AlphaClifford can be applied to a variety of tasks, including general Clifford circuits optimization, Clifford transpilation to ensure compatibility with a given hardware connectivity map, and as a component in a pipeline for logical synthesis, from a general initial circuit, to the final approximated version in Clifford+T.

the circuit compilation problem as a tree-based search over the space of symplectic matrices, with the aim of finding the shortest path from an initial matrix to the identity, allowing for a compact reconstruction of the corresponding Clifford circuit from the set gate $\{H, S, CNOT\}$.

To demonstrate the broad applicability of our proposed method, we evaluate it on three different tasks: unconstrained Clifford optimization, Clifford transpilation to a target hardware connectivity map, and as a refinement step in a full logical synthesis pipeline.

We show that our model achieves comparable or higher performance with respect to a variety of different techniques, including up to 56% total gate reduction compared to the Aaronson-Gottesman algorithm [28] for the general Clifford optimization task, and up to 20% compared to the RL-based Clifford transpiler from [26] on the task of transpilation. Our results suggest that AlphaClifford can be applied efficiently in different settings.

The paper is organized as follows. In Section II we provide a brief introduction to the symplectic representation of Clifford circuits. In Section III we discuss related works. In Section IV we present our method. Experimental design and results are discussed in Section V. Finally, in Section VI we provide some final considerations and suggest possible directions for further works.

II. SYMPLECTIC REPRESENTATION OF CLIFFORD CIRCUITS

A. Stabilizer Formalism and Tableau Representation

The n -qubit Pauli group $\mathcal{P}_n$ consists of all n -fold tensor products of the Pauli matrices $\{I, X, Y, Z\}$ together with the multiplicative phase factors $\{\pm 1, \pm i\}$. The Clifford group $\mathcal{C}_n$ is defined as the normalizer of the Pauli group within the unitary group $U(2^n)$. Specifically, for any Clifford operator $C \in \mathcal{C}_n$ and any Pauli operator $P \in \mathcal{P}_n$, the conjugation $CPC^\dagger$ yields another operator in $\mathcal{P}_n$. The group $\mathcal{C}_n$ is finitely generated by the Hadamard (H), Phase (S), and Controlled-NOT ($CNOT$) gates.

By the Gottesman-Knill theorem [29], Clifford circuits can be efficiently simulated on classical hardware. The standard data structure for this is the tableau representation introduced by Aaronson and Gottesman [28]. The state of an n -qubit Clifford circuit is completely characterized by tracing the evolution of n destabilizer generators and n stabilizer generators. In this formalism, the circuit is represented as a $2n \times (2n+1)$ binary matrix $\mathcal{T}$ over the Galois field $\mathbb{F}_2$:

$$\mathcal{T} = [\mathcal{X} \quad \mathcal{Z} \mid \mathbf{r}], \quad (1)$$

where $\mathcal{X}$ and $\mathcal{Z}$ are $2n \times n$ Boolean matrices denoting the presence of X and Z Pauli operators, respectively. The $2n$ -dimensional column vector $\mathbf{r}$ contains Boolean phase bits indicating whether the overall sign of each respective generator is $+1$ ($r_i = 0$) or -1 ($r_i = 1$).

As a simple example, consider a single-qubit system. The initial tableau corresponding to the identity circuit is

$$\mathcal{T}_I = \left[\begin{array}{cc|c} 1 & 0 & 0 \\ 0 & 1 & 0 \end{array} \right],$$

where the first row represents the destabilizer generator X and the second row represents the stabilizer generator Z .

Applying a Hadamard gate H exchanges X and Z , resulting in the tableau

$$\mathcal{T}_H = \left[\begin{array}{cc|c} 0 & 1 & 0 \\ 1 & 0 & 0 \end{array} \right].$$

This illustrates how Clifford operations act linearly on the $(\mathcal{X}, \mathcal{Z})$ components of the tableau by transforming Pauli generators. If we further apply an X gate, the Pauli structure remains unchanged, but the phase of the Z generator is flipped:

$$\mathcal{T}_{XH} = \left[\begin{array}{cc|c} 0 & 1 & 0 \\ 1 & 0 & 1 \end{array} \right].$$

Thus, while the matrices $\mathcal{X}$ and $\mathcal{Z}$ encode the Pauli action of the circuit, the vector $\mathbf{r}$ captures the accumulated phase information.

B. Symplectic Matrix Representation

While the phase vector $\mathbf{r}$ is required to track the exact quantum state (including Pauli phases), the fundamental entangling structure and basis transformations induced by a Clifford circuit are entirely captured by the $2n \times 2n$ binary matrix $M = [\mathcal{X} \ \mathcal{Z}]$.

To preserve the commutation relations of the Pauli group, the matrix M must satisfy the symplectic condition over $\mathbb{F}_2$:

$$M\Omega M^T = \Omega \pmod{2}, \quad (2)$$

where Ω is the $2n \times 2n$ block matrix defined as:

$$\Omega = \begin{bmatrix} 0 & I_n \\ I_n & 0 \end{bmatrix}, \quad (3)$$

and I_n is the $n \times n$ identity matrix. Consequently, M is an element of the symplectic group $Sp(2n, \mathbb{F}_2)$.

Crucially, there exists a surjective group homomorphism $\phi : \mathcal{C}_n \rightarrow Sp(2n, \mathbb{F}_2)$ that maps a Clifford unitary to its corresponding symplectic matrix. The kernel of this homomorphism is precisely the Pauli group $\mathcal{P}_n$ (ignoring global phases).

C. Implications for Circuit Synthesis

In our context of synthesizing Clifford circuits, the homomorphism ϕ provides a powerful mathematical abstraction. If an RL agent restricts its objective to finding a sequence of $\{H, S, \text{CNOT}\}$ gates that yields a symplectic matrix M_{syn} matching a target symplectic matrix M_{target} , the synthesized circuit C_{syn} and the target circuit C_{target} are guaranteed to belong to the same coset of the Pauli group. Mathematically, this establishes an equivalence up to a Pauli layer:

$$C_{\text{target}} \equiv P \cdot C_{\text{syn}}, \quad (4)$$

where $P \in \mathcal{P}_n$. By tracking the evolution of the phase vector $\mathbf{r}$ alongside the synthesis process, the residual Pauli correction

P can be computed efficiently in $\mathcal{O}(n^2)$ time. Therefore, mapping the RL state space to $Sp(2n, \mathbb{F}_2)$ rather than the full Clifford group $\mathcal{C}_n$ reduces the search space by a factor of 2^{2n} , simplifying the learning process.

III. RELATED WORK

The problem of quantum circuit synthesis and compilation is extremely important for mitigating hardware limitations of current devices, and the literature addressing this challenge is extensive [30].

Within the context of Clifford circuits, the historical baseline is the well-known Aaronson-Gottesman algorithm [28], which demonstrated that this class of circuits can be efficiently simulated and synthesized in polynomial time with gates from $\{H, S, \text{CNOT}\}$.

However, standard approaches usually generate sub-optimal circuits with respect to the number of gates. To address this, various advanced heuristic methods have been proposed.

For instance, the approaches described in [31] (such as A^* , greedy algorithms, and Volanto [32]) leverage a richer generating set to reduce the gate count in the circuit. Another approach is the one proposed in [33], where Clifford-specific template matching is combined with symbolic peephole optimization, obtaining relevant reductions in CNOT count.

It is also worth noting that the literature offers solutions for the optimal and exact syntheses of Clifford circuits [34], [35]. However, finding the absolute minimum sequence of gates generally requires exponential time, drastically limiting the scalability of these methods to a very small number of qubits and rendering them unsuitable for practical, large-scale applications.

Recently, many Reinforcement Learning (RL) based solutions have been proposed to address tasks related to circuit synthesis, optimization, and compilation, including state preparation [24], quantum architecture search [36], and “pre-compilation” in logical synthesis pipelines in Q-PRESYN [37].

The synthesis of Clifford circuits is particularly well-suited for RL applications: the algebraic properties of these circuits provide a compact and highly efficient state representation, usually as a matrix of polynomial size. Initially, many works were based on **model-free** algorithms, such as Proximal Policy Optimization [38] (PPO), which learn directly by interacting with the environment without building an internal model of the dynamics.

Notable among these is the recent work [26], which leverages RL for the routing and synthesis of Clifford circuits, yielding transpilers capable of significantly outperforming traditional heuristics like SABRE [39]. Another impactful model-free application is provided in [22] in the context of quantum circuit discovery for fault-tolerant logical state preparation, where RL was used to discover entirely new protocols that surpass the efficiency of manually designed circuits. Additionally, in [25] the authors propose a PPO-based approach to optimize CNOT count in linear reversible circuits.

Despite the significant successes of model-free methods, recent developments demonstrate that when the mathematical

structure of the problem allows it, *model-based* algorithms, usually based on Monte Carlo Tree Search [40] (MCTS), offer vastly superior capabilities for exploring complex combinatorial spaces.

In this regard, AlphaTensor-Quantum [19] paved the way by demonstrating how the CNOT+ T component of a circuit can be optimized using search techniques originally designed for board games [41]. Similarly, AlphaCNOT [27] provided a model to address the problems of CNOT optimization in both unconstrained and constrained (i.e., depending on a connectivity map) settings.

Finally, another relevant contribution is *QuSynth* [42], in which the task of stabilizer state preparation is reformulated by using graph-state representations, and addressed by combining RL with MCTS for structured look-ahead.

Our work, **AlphaClifford**, builds on these developments by bringing the strengths of search-informed reinforcement learning to the synthesis of general Clifford circuits. In particular, we draw inspiration from the AlphaTensor and AlphaCNOT frameworks, adapting their core ideas to exploit the algebraic structure of Clifford operations and further advance the state of the art in this domain.

IV. ALPHACLIFFORD

A popular class of Reinforcement Learning (RL) approaches is classified as *model-free*, i.e., they learn directly from interactions with the environment and do not leverage an explicit model of its dynamics. Model-free approaches, such as Proximal Policy Optimization [38] (PPO), rely on sequences of state–action–reward transitions to iteratively update the policy via gradient-based optimization.

Conversely, *model-based* RL algorithms construct a representation of the environment to simulate future outcomes [43]. Within complex combinatorial spaces, such as the set of symplectic matrices, their capacity for structured and efficient exploration enables them to obtain improved performance over both classical heuristics and model-free algorithms.

Following the latter principle, and inspired by recent applications such as AlphaTensor [19] and AlphaCNOT [27], we propose **AlphaClifford**, a novel model-based approach for exploring the combinatorial space of symplectic matrices associated with Clifford circuits.

Our method, illustrated in Fig. 1, is organized into the following steps:

- First, we model the space of symplectic matrices as a search tree (Fig. 1a), where each node represents a specific symplectic matrix and directed edges correspond to the application of Clifford gates. In this representation, two nodes are connected if the corresponding matrices can be reached through a Clifford gate operation.
- Second, we train a model to learn the structure of this space by predicting sequences of gate operations that transform the initial symplectic matrix into the target one. The training procedure, structured after the Monte Carlo Tree Search pattern [40], is detailed in Section IV-C.

- Third, once trained, the model is used to synthesize Clifford circuits (Fig. 1b) by generating a new sequence of gates from a target circuit. The generated sequence of Clifford gates will have the same symplectic matrix as the one associated to the target circuit.

A. Modeling the problem of symplectic matrices

The search space of our problem, represented by the set of symplectic matrices, is explored through a tree-based approach. For a given problem dimension n (corresponding to the number of qubits), given a target Clifford circuit C represented by its symplectic matrix $M_C \in Sp(2n, \mathbb{F}_2)$, and possibly a connectivity map $\mathcal{M}$, our aim is to determine a sequence of Clifford gates equivalent to C .

To this end, we construct a search tree as follows:

- The root node represents the identity matrix I_{2n} , corresponding to the empty circuit;
- Each node N is labeled by a symplectic matrix M and has one child for each admissible Clifford operation $g \in \{H_i, S_i, \text{CNOT}_{ij} : i, j \in \{1, \dots, n\}\}$ (subject to $\mathcal{M}$). The child node is labeled by the matrix obtained by applying g to M , i.e., by the product of M with the matrix corresponding to g ;
- The terminal nodes (i.e., the leaves) are labeled by the target symplectic matrix M_C .

In the unconstrained setting, each node has $\Theta(n^2)$ children due to the possible CNOT operations between pairs of qubits, along with single-qubit gates. In the case of limited connectivity constraints, only a subset of 2-qubit operations is allowed.

B. Inverting the representation: from the target to the identity

Training an RL agent with the forward representation described in Section IV-A would require providing, at each step k , both the current symplectic matrix M_k (encoding the partial circuit $g_1, \dots, g_k$) and the fixed target matrix M_C . This increases the input dimensionality and, moreover, the target matrix contributes little information during an episode, as it remains constant. To avoid this, we adopt an inverted formulation that embeds the target directly into the state representation.

At step k , the observation is defined as

$$M_C M_k^{-1},$$

where M_k^{-1} is the inverse of the current symplectic matrix over $\mathbb{F}_2$, corresponding to the inverse circuit. Under this representation, synthesizing the target Clifford circuit is equivalent to reaching the identity:

$$M_C M_C^{-1} = I_{2n}.$$

Hence, each episode begins in the state M_C , and the agent's task is to reach the identity matrix. This turns the problem into a goal-reaching task with a fixed terminal state and allows us to provide a single observation matrix to the model, rather than a pair (M_k, M_C) .

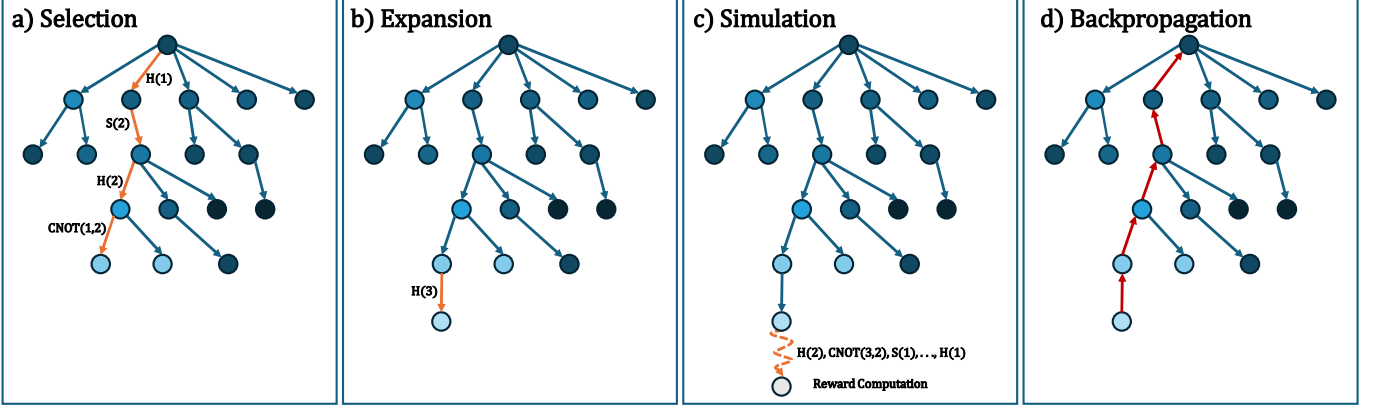


Fig. 2: Overview of the Monte Carlo Tree Search (MCTS) phases for Clifford circuit synthesis. The search space is explored by iteratively executing four distinct stages: (a) **Selection**: The tree is traversed from the root (representing the target symplectic matrix C) to a leaf node. Actions, corresponding to Clifford gates H , S , and $CNOT$, are chosen by balancing exploration and exploitation using the UCT formula. (b) **Expansion**: When reaching a leaf node, the tree is expanded by the addition of one or more child nodes, each representing the state resulting from a possible Clifford operation. (c) **Simulation**: A rollout is performed from the newly expanded node based on the policy network. A sequence of gates is sampled until a terminal state is reached, at which point the evaluation of the value network is computed based on the proximity to the identity matrix. (d) **Backpropagation**: The reward obtained during simulation is propagated upward to the root, updating the value estimates and visit counts for all nodes along the selected path to refine future selection steps.

When a gate g_t is applied in the forward construction of the circuit, the corresponding update in the inverted representation must use the adjoint action. Since

$$M_t = M_{g_t} M_{t-1} \implies M_t^{-1} = M_{t-1}^{-1} M_{g_t}^\dagger,$$

the state transition in the inverted formulation becomes

$$M_C M_{t-1}^{-1} \longrightarrow M_C \left(M_{t-1}^{-1} M_{g_t}^\dagger \right).$$

Thus, although the agent selects forward Clifford operations, the evolution of the internal representation proceeds by right-multiplication with the corresponding symplectic matrices, consistently reflecting the reversed dynamics.

C. Monte Carlo Tree Search Framework

We adopt a Monte Carlo Tree Search [40] (MCTS) strategy to efficiently explore the space of symplectic matrices associated with Clifford circuits (see Fig. 2). The procedure is structured into four main phases:

- 1) **Selection**: Starting from the root node corresponding to the target symplectic matrix M_C , the tree is traversed by iteratively selecting actions (i.e., Clifford gates in $\{H, S, CNOT\}$) according to a tree policy. In particular, we adopt the Upper Confidence Bound for Trees (UCT) criterion [43], which selects the action a maximizing

$$\text{UCT}(a) = Q(s, a) + c \sqrt{\frac{\log N(s)}{N(s, a)}}, \quad (5)$$

where $Q(s, a)$ is the estimated value of taking action a in state s , $N(s)$ is the visit count of node s , $N(s, a)$ is the number of times action a has been selected from s ,

and c is an exploration constant. This criterion balances exploration of less-visited actions and exploitation of high-value ones;

- 2) **Expansion**: When a non-terminal node with unexplored actions is reached, the tree is expanded by applying a new admissible Clifford operation, thus generating a new child node. A node is terminal if it corresponds to the identity matrix or if other stopping criteria are met (e.g., maximum depth). In the presence of connectivity constraints, only allowed $CNOT$ operations are considered;
- 3) **Simulation**: From the expanded node, a rollout is performed by sampling a sequence of Clifford gates according to the policy network until a terminal node is reached. The resulting trajectory is then processed by the value function to estimate the quality of the new node;
- 4) **Backpropagation**: The outcome of the simulation is propagated back along the selected path, updating visit counts and value estimates for each node involved in the traversal.

D. Training Details

The training consists of constructing a different random target matrix for each episode, and asking the model to “reach” the identity matrix starting from the inverse of the target with the fewest amount of moves. Each “move”, corresponding to a *step* in the environment, represents the application of a feasible gate to the current circuit. Feasible gates are H , S , and $CNOT$ s, possibly with constraints based on topology connectivity. An episode finishes after reaching the identity,

or when the agent perform $n_{\max}$ moves without reaching the identity.

We adopt a naive reward function to guide the learning process. At each step a penalty of $-1/\sqrt{n}$ is applied to limit long sequences and a terminal reward of $+n$ is assigned when reaching the identity matrix. This “non-informed” strategy is fundamental in our approach: without relying on problem-specific heuristics, the model itself naturally learns to find shorter decompositions. In particular, the training objective implicitly minimizes the total number of applied gates, which also implies a reduction of CNOTs. Note that different rewards could be used to minimize different target functions: for example, we can select a higher penalty to reduce the CNOT count.

We employ a linear curriculum learning strategy [44] based on the (expected) difficulty of the instances. In practice, we generate each target symplectic matrix by selecting random k Clifford gates, where k is parameter we can tune during training.

More in detail, we divide our training in tranches. Starting from $k = 1$, we train our model on a tranche of difficulty k . At the end of the tranche, we increase k if the model reached a solving rate of at least 95%, i.e., the model is able to correctly reproduce most matrices. We stop the training after the max difficulty of $k_{\max} = 4n^2$ is reached.

E. Inference

Once the model has been trained to synthesize symplectic matrices of size $2n \times 2n$, optionally subject to hardware connectivity constraints, it can be deployed for inference. Specifically, given a target Clifford circuit C_{tar} characterized by a symplectic matrix M_{tar} , our model generates a sequence of gates forming a synthetic circuit C_{syn} that yields the identical symplectic matrix. Consequently, $M_{\text{tar}} = M_{\text{syn}}$, which establishes that the target and synthetic circuits are equivalent up to a Pauli operator (i.e., $C_{\text{tar}} \equiv PC_{\text{syn}}$ for a pauli P).

The complete inference procedure for a given target circuit proceeds as follows: (i) efficiently compute its target symplectic matrix M_{tar} ; (ii) query the trained model to synthesize a gate sequence corresponding to this matrix; (iii) efficiently compute the phase discrepancy—namely, the signs of the stabilizers and destabilizers—between the target and synthetic circuits; and (iv) apply the appropriate X and Z gates to correct the phase, yielding a fully equivalent circuit. A visual overview of this pipeline is provided in Fig. 1b.

F. Applications

We identify three primary applications where our model can be efficiently deployed: Clifford optimization (Fig. 1c, top), Clifford transpilation (Fig. 1c, middle), and as a subroutine within a standard logical synthesis pipeline (Fig. 1c, bottom). These settings are described below and are experimentally demonstrated in Sections V-A, V-B, and V-C, respectively.

1) *Optimization*: A direct application of our framework is Clifford circuit optimization. Given a target Clifford circuit, the objective is to find an equivalent representation

that minimizes the total number of two-qubit gates. More broadly, the model can be tailored during training to minimize alternative target metrics, such as overall circuit depth. Beyond full-circuit synthesis, AlphaClifford can also be employed as a peephole optimizer [45] for Clifford circuits involving a larger number of qubits. Starting, for example, from a circuit synthesized using the Aaronson–Gottesman algorithm, one can extract contiguous subcircuits whose support contains at most n qubits, resynthesize each block using the corresponding n -qubit AlphaClifford model, and replace the original block whenever the selected cost is reduced. By iteratively applying this procedure, models trained on relatively small problem sizes can be used to optimize substantially larger circuits, although the resulting improvements are local and do not imply global optimality.

2) *Transpilation*: We further evaluate our approach on the task of quantum circuit transpilation. Given n qubits and a hardware connectivity map—represented as a graph where nodes correspond to physical qubits and edges denote allowable direct interactions—the goal of transpilation is to transform a logical circuit into an equivalent physical circuit that strictly adheres to these connectivity constraints. This is usually achieved by mapping logical qubits to physical qubits and inserting additional gates (e.g., SWAP routing) to ensure all two-qubit operations occur between adjacent nodes. Addressing this task is critical for near-term quantum execution, as many hardware platforms (such as superconducting architectures [46]) only permit multi-qubit operations between physically coupled qubits.

Similar to circuit optimization, the objective is to find a valid transpilation that minimizes a specific cost function, typically the two-qubit gate count or the overall circuit depth. Furthermore, for realistic hardware deployments, the cost function can be expanded to incorporate device-specific characteristics, such as penalizing the use of qubits or couplers with high error rates. To accomplish this within our framework, the model’s action space during training is constrained to the specific connectivity map of the target hardware connectivity map.

3) *Optimizing Logical Synthesis Pipeline*: A crucial task in fault-tolerant quantum computing is logical synthesis, which involves approximating a target unitary using a discrete, universal gate set such as Clifford+T [8]. Unlike pure Clifford synthesis, optimal exact synthesis over universal gate sets is a notoriously difficult problem, with worst-case computational complexity scaling exponentially with respect to the number of qubits [47], [48]. To address this limitation, compilers typically employ a “peephole” synthesis strategy, decomposing the target circuit into much smaller, tractable sub-blocks (typically acting on two or three qubits) [33].

A standard logical synthesis pipeline generally operates in successive stages. First, a pre-processing step can be applied to the logical circuit. This may involve decomposing large multi-qubit gates into smaller two- or three-qubit instances (e.g., via recursive Cartan decomposition [49], [50]) or performing structural pre-synthesis edits to make the circuit more amenable to compilation [37]. Next, a local synthesis

algorithm compiles each of these individual sub-blocks into a Clifford+T representation.

Our framework can be integrated into the final stage of this pipeline as a post-synthesis optimizer. Once the full circuit has been compiled into Clifford+T, we can extract the largest contiguous blocks of purely Clifford gates. Our model can then be applied directly to these extracted Clifford sub-circuits to find equivalent representations that minimize the total gate count, effectively compressing the circuit without altering the expensive T-gate layout.

While our experimental evaluation in Section V-C demonstrates this capability on the relatively small Clifford blocks generated by standard peephole synthesis, our approach can be applied more in general in any kind of compilation task involving large blocks of consecutive Clifford gates.

V. EXPERIMENTAL RESULTS

In the following, we describe the experimental evaluations and the obtained results for the tasks of Clifford Optimization (Section V-A), Clifford Transpilation (Section V-B), and optimization in the logical synthesis pipeline (Section V-C).

Both policy and value networks share the same architecture, i.e., a 12-layer fully connected network with 256 hidden units per layer. We structured the net using 4 residual blocks with skip connections every three layers. We trained a different model for each number of qubits n , and for each selected connectivity map for the transpilation task. The training was executed in tranches of different complexity (computed as the number of generating gates) as described in Section IV-D, and increasing once the solving threshold of 95% is achieved for the tranche, up to a complexity of $4n^2$.

A. Clifford Optimization

As a first task, we evaluate the ability of our model to efficiently synthesize sufficiently complex Clifford circuits. This problem can be reformulated as a Clifford optimization task: we start from a random Clifford circuit generated by `qiskit` [51], and we synthesize an equivalent circuit with gates from $\{H, S, \text{CNOT}\}$.

For n qubits with $n \in \{3, 4, 5, 6, 7\}$, we evaluate our model on 100 Clifford circuits involving up to $4n^2$ gates. These experiments evaluate full synthesis, in which the complete target circuit is provided to a single model. Note that, as discussed in Section IV-F, the same models can be applied as local peephole optimizers to blocks of larger Clifford circuits.

To evaluate our approach, we consider both the average total number of gates, and the number of two-qubit gates (in our case, CNOTs) required to synthesize a circuit. We evaluate our model both one shot, and after 10 repetitions, i.e. we sample our model 10 times, and select the circuit with lowest amount of gates. We denote the latter as `AlphaClifford10`. Our results are reported in Table I.

As comparison, we consider first the well-known Aaronson-Gottesman algorithm [28], as implemented in `qiskit` [51]), and then three different heuristic models (A^* , greedy, and Volanto [32]), as provided in [31]. These three methods

provide a decomposition in a block of SWAP gates, followed by a block of one and two qubit *transvection* gates [52]. Given a n -qubit Pauli operator P , the corresponding transvection can be defined as

$$T_P := \exp\left(\frac{i\pi}{4}(I_n - P)\right). \quad (6)$$

Note that by allowing a richer gate set provides an unfair comparison versus our approach that is constrained to the less expressive set $\{H, S, \text{CNOT}\}$. Nevertheless, we include this evaluation, where the SWAP layer is first decomposed in three CNOTs. For completeness, we also consider the CNOT count obtained by the same three methods when the obtained circuits are decomposed in $\{H, S, \text{CNOT}\}$ by the default transpiler provided by `qiskit`. Finally, we report the same metrics obtained by the `qiskit greedy` method, and by the well-known `stim` software for stabilizer simulation [53].

Approach	Problem Size (n)				
	3	4	5	6	7
Total Gates					
Aaronson-Gottesman	19.99	25.89	38.05	51.37	65.61
Qiskit greedy	13.57	23.03	34.75	47.77	62.18
Stim	21.27	36.28	54.70	72.82	96.39
A^* (with transv.)	12.97	19.56	26.54	35.04	43.15
Greedy (with transv.)	13.03	19.40	27.42	36.08	45.61
Volanto (with transv.)	12.95	19.93	28.65	38.12	48.14
A^* (H, S, CNOT)	19.49	33.47	49.90	68.33	89.47
Greedy (H, S, CNOT)	19.17	33.21	51.44	72.16	98.29
Volanto (H, S, CNOT)	20.26	37.30	59.25	87.19	117.57
AlphaClifford	8.85	13.90	20.66	29.56	40.05
AlphaClifford₁₀	8.78	13.39	19.49	27.11	36.75
2-Qubit Gates					
Aaronson-Gottesman	3.65	10.41	16.17	22.43	28.59
Qiskit greedy	4.35	8.01	13.01	18.79	24.95
Stim	7.29	13.20	20.32	29.49	40.35
A^* (with transv.)	6.34	10.63	15.72	21.39	27.32
Greedy (with transv.)	6.28	10.60	16.55	22.71	29.86
Volanto (with transv.)	6.38	11.16	17.36	24.38	32.62
A^* (H, S, CNOT)	8.84	15.38	23.10	31.89	41.59
Greedy (H, S, CNOT)	8.78	15.39	24.39	34.19	46.28
Volanto (H, S, CNOT)	9.19	17.04	27.37	39.85	54.23
AlphaClifford	4.25	7.50	11.85	17.47	25.53
AlphaClifford₁₀	4.06	6.96	10.98	15.82	23.06

TABLE I: Average number of total and 2-Qubit gates for different optimization methods over varying problem sizes. Best results for each n are highlighted in bold.

We observe that our model achieves the lowest amount of total gates, and two qubit gates, in most cases. Note that all methods were able to synthesize correctly each instance, with the exception of Greedy method that was not able to synthesize a few instances of larger dimensions (in this case, the average reported in Table I is considered only on the solved instances).

We stress that the advantage compared to the methods in [31] is obtained with a less expressive gate. We expect our

method to obtain even better performance when allowing more expressive gate sets.

B. Transpilation

To evaluate our model, for each number of qubits and connectivity map, we generate a random Clifford circuit without connectivity constraints. We then use a model trained on the specific connectivity map to generate an equivalent circuit by using the gate set $\{H, S, \text{CNOT}\}$, where a CNOT can be selected only between pairs of qubits allowed by the connectivity map.

Following the procedure in [26], we select 6 connectivity maps from 3 to 6 qubits (see Fig. 3 for additional information). As before, we provide total gate count, and CNOT count. As a comparison, we evaluate the RL-based Clifford transpiler by `qiskit`, available at [54]. Both methods are sampled once and 10 times, to take into account the stochasticity of RL-based approaches. Results are provided in Table II. We observe that in general, our model provides consistently a lower total gate counts than the other approach. However, it also obtains a slightly higher number of CNOTs, suggesting that a reward function tailored on the number of two-qubit gates instead of total gates could be beneficial in this specific setting.

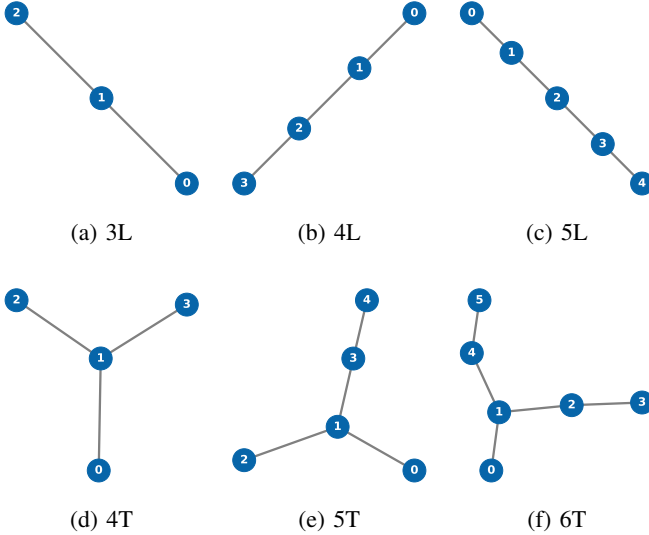


Fig. 3: Hardware connectivity maps utilized for benchmarking: Line graphs (a-c) and T-shape graphs (d-f). Vertices represent physical qubits and edges represent available coupling map connections.

C. Full Logical synthesis pipeline

Finally, we evaluate our proposed method as a component of a logical synthesis pipeline. To do so, we first generate 10 random circuits for each number of qubits $n \in \{5, 10, 15, 20, 25\}$. Following the approach in [37], we consider the pipeline:

- Starting from the representation of the initial circuit, we decompose each 2 qubit gates in a block of CNOTs and single qubit gates with the KAK decomposition [49];

- Then, we decompose each single qubit unitary in up to three $R_z(\theta)$ rotations (with additional X and H gates). Each rotation is then decomposed in $\{H, S, T\}$ with the GRIDSYNTH algorithm [55] with a tolerance of $\varepsilon = 0.01$;
- Finally, we search for blocks of adjacent Clifford circuits. For each block of at least 2 qubits, we use AlphaClifford to synthesize an optimized version.

In addition, we evaluate the same pipeline with the use of Q-PRESYN [37] to optimize the number of T gates with a sequence of 2 qubit merges. In particular, we apply a greedy optimizer to evaluate the largest sequence of gates that can be merged together, with the aim of reducing the number of resulting T gates after applying the KAK decomposition followed by GRIDSYNTH.

In Table III we present the Clifford gate reduction, with the default `qiskit` transpiler as a comparison. Note that in this specific setting, due to the particular structure of the Clifford circuits (that are already optimal in the number of CNOT gates), the algorithms Aaronson-Gottesman and the ones from [31] did not obtain any reduction in total number of gates. Finally, note that the considered pipeline applied to random circuits leads to mostly 2 and 3 qubit Clifford blocks, therefore allowing for a small reduction of gates. We expect different choices of logical synthesis algorithms and different circuit families to provide higher reductions.

VI. CONCLUSION

In this work, we introduced AlphaClifford, a novel model-based reinforcement learning framework designed to address the complex combinatorial challenges of Clifford circuit synthesis and transpilation. By formulating the compilation process as a Monte Carlo Tree Search over the algebraic space of symplectic matrices, AlphaClifford consistently discovers highly optimized gate sequences.

Our experimental evaluations demonstrate that AlphaClifford achieves superior or comparable results to state-of-the-art methods across a variety of settings. For unconstrained optimization, it successfully minimizes both total and two-qubit gate counts compared to standard heuristics, even when restricted to a less expressive gate set. Furthermore, we showed its adaptability to hardware-constrained transpilation and its practical utility as a post-synthesis optimizer within a logical synthesis pipeline. Notably, these results were obtained using a single base architecture and a naive reward function, underscoring the generality and robustness of our technique.

Ultimately, these results highlight the potential of model-based RL to transcend the limitations of traditional heuristic-driven compilers, advancing the trajectory established by frameworks like AlphaTensor [19] and AlphaCNOT [27]. As quantum hardware continues to scale toward the era of quantum utility, computationally intelligent approaches like AlphaClifford will be critical for bridging the gap between high-level logical algorithms and the strict physical constraints of near-term and future fault-tolerant quantum devices.

We highlight several promising directions for future work. First, extending the action space to include more expres-

Approach	L Maps						T Maps					
	3L		4L		5L		4T		5T		6T	
	Tot.	CNOT	Tot.	CNOT	Tot.	CNOT	Tot.	CNOT	Tot.	CNOT	Tot.	CNOT
RL-Qiskit ₁	14.52	4.91	26.80	10.51	36.95	16.78	26.38	9.60	33.63	15.15	50.85	24.41
RL-Qiskit ₁₀	12.19	4.60	22.90	9.54	33.31	16.08	21.58	8.43	30.92	14.29	46.11	22.74
AlphaClifford ₁	9.83	5.15	17.49	10.66	29.07	18.45	16.16	9.51	26.27	16.82	42.35	27.49
AlphaClifford ₁₀	9.81	4.99	17.08	10.08	27.58	17.31	15.76	8.96	25.04	15.56	38.84	25.10

TABLE II: Performance of RL-Qiskit [54] and AlphaClifford for the transpilation task. Results report average total (Tot.) and CNOT gate counts for L and T connectivity maps. Best values for metric are reported in bold.

n	Initial Compiled		Qiskit Transpile		AlphaClifford	
	Clifford (k)	T-count (k)	Clifford (k)	% _{red}	Clifford (k)	% _{red}
Without Pre-compilation						
5	1.24	0.76	1.23	0.36	1.21	2.44
10	5.17	3.14	5.16	0.13	5.05	2.39
15	10.27	6.23	10.26	0.09	10.01	2.53
20	20.26	12.28	20.24	0.14	19.76	2.50
25	29.59	17.92	29.55	0.14	28.88	2.43
With Pre-compilation						
5	1.09	0.66	1.08	0.35	1.06	2.75
10	4.90	2.97	4.90	0.17	4.78	2.46
15	9.98	6.06	9.97	0.15	9.72	2.57
20	19.77	11.98	19.75	0.12	19.28	2.50
25	29.18	17.66	29.12	0.20	28.45	2.49

TABLE III: Clifford synthesis performance: comparison of Clifford gate counts (in thousands) and reduction percentages relative to the original synthesized circuit. Results are grouped by whether pre-synthesis [37] was applied.

sive operations, such as transvections, could further enhance compilation efficiency. Second, while our generalized reward function successfully minimized total gate counts, designing tailored reward structures—such as heavily penalizing two-qubit gates to improve specific transpilation connectivity maps or incorporating error-aware routing—could yield even better hardware-specific solutions. Finally, a further direction is to investigate systematic block-selection and iterative peephole strategies for applying the models developed here to large-scale Clifford circuits.

VII. ACKNOWLEDGEMENTS

DLB thanks Gianluca Macrì for his support in preparing the figures and diagrams of this work. JC is supported by FSE/FVG PhD Grant on “Computer Science and Artificial Intelligence” (CUP G23C25000620008).

This work has been partially supported by INdAM-GNCS project *Algebra lineare quantistica, state preparation e compilazione di circuiti quantistici* (CUP E53C25002010001) and by the regional project QUASAR-FVG *Calcolo e simulazione quantistica: sviluppo, applicazioni e ricerca in Friuli Venezia Giulia* (CUP G23C25001510002).

REFERENCES

- [1] Y. Kim, A. Eddins, S. Anand, K. X. Wei, E. Van Den Berg, S. Rosenblatt, H. Nayfeh, Y. Wu, M. Zaletel, K. Temme, and A. Kandala, “Evidence for the utility of quantum computing before fault tolerance,” *Nature*, vol. 618, no. 7965, pp. 500–505, Jun. 2023.
- [2] Y. Alexeev, V. S. Batista, N. Bauman, L. Bertels, D. Claudino, R. Dutta, L. Gagliardi, S. Godwin, N. Govind, M. Head-Gordon, M. R. Hermes, K. Kowalski, A. Li, C. Liu, J. Liu, P. Liu, J. M. García-Lastra, D. Mejía-Rodríguez, K. Mueller, M. Otten, B. Peng, M. Raugas, M. Reiher, P. Rigor, W. J. Shaw, M. van Schilfgaarde, T. Vegge, Y. Zhang, M. Zheng, and L. Zhu, “A perspective on quantum computing applications in quantum chemistry using 25–100 logical qubits,” *J. Chem. Theory Comput.*, vol. 21, no. 22, pp. 11 335–11 357, 2025.
- [3] A. Di Meglio, K. Jansen, I. Tavernelli, C. Alexandrou, S. Arunachalam, C. W. Bauer, K. Borras, S. Carrazza, A. Crippa, V. Croft, R. de Putter, A. Delgado, V. Dunjko, D. J. Egger, E. Fernández-Combarro, E. Fuchs, L. Funcke, D. González-Cuadra, M. Grossi, J. C. Halimeh, Z. Holmes, S. Kühn, D. Lacroix, R. Lewis, D. Lucchesi, M. L. Martinez, F. Meloni, A. Mezzacapo, S. Montangero, L. Nagano, V. R. Pascuzzi, V. Radescu, E. R. Ortega, A. Roggero, J. Schuhmacher, J. Seixas, P. Silvi, P. Spentzouris, F. Tacchino, K. Temme, K. Terashi, J. Tura, C. Tüysüz, S. Vallecorsa, U.-J. Wiese, S. Yoo, and J. Zhang, “Quantum computing for high-energy physics: State of the art and challenges,” *PRX Quantum*, vol. 5, p. 037001, Aug 2024.
- [4] A. Bochkarev, R. Heese, S. Jäger, P. Schiewe, and A. Schöbel, “Quantum computing for discrete optimization: A highlight of three technologies,” *Eur. J. Oper. Res.*, vol. 329, no. 3, pp. 747–766, 2026.
- [5] D. Herman, C. Googin, X. Liu, Y. Sun, A. Galda, I. Safro, M. Pistoia, and Y. Alexeev, “Quantum computing for finance,” *Nat. Rev. Phys.*, vol. 5, no. 8, pp. 450–465, Jul. 2023.
- [6] Google Quantum AI, R. Acharya, I. Aleiner, R. Allen, T. I. Andersen, M. Ansmann, F. Arute, K. Arya, A. Asfaw, J. Atalaya, R. Babbush, D. Bacon, J. C. Bardin, J. Basso, A. Bengtsson, S. Boixo, G. Bortoli, A. Bourassa, J. Bovaird, L. Brill, M. Broughton, B. B. Buckley, D. A. Buell, T. Burger, B. Burkett, N. Bushnell, Y. Chen, Z. Chen, B. Chiaro, J. Cogner, R. Collins, P. Conner, W. Courtney, A. L. Crook, B. Curtin, D. M. Debroy, A. Del Toro Barba, S. Demura, A. Dunsworth, D. Eppens, C. Erickson, L. Faoro, E. Farhi, R. Fatemi, L. Flores Burgos, E. Forati, A. G. Fowler, B. Foxen, W. Giang, C. Gidney, D. Gilboa, M. Giustina, A. Grajales Dau, J. A. Gross, S. Habegger, M. C. Hamilton, M. P. Harrigan, S. D. Harrington, O. Higgott, J. Hilton, M. Hoffmann, S. Hong,

- T. Huang, A. Huff, W. J. Huggins, L. B. Ioffe, S. V. Isakov, J. Iveland, E. Jeffrey, Z. Jiang, C. Jones, P. Juhas, D. Kafri, K. Kechedzhi, J. Kelly, T. Khattar, M. Khezri, M. Kieferová, S. Kim, A. Kitaev, P. V. Klimov, A. R. Klots, A. N. Korotkov, F. Kostritsa, J. M. Kreikebaum, D. Landhuis, P. Laptev, K.-M. Lau, L. Laws, J. Lee, K. Lee, B. J. Lester, A. Lill, W. Liu, A. Lochlarla, E. Lucero, F. D. Malone, J. Marshall, O. Martin, J. R. McClean, T. McCourt, M. McEwen, A. Megrant, B. Meurer Costa, X. Mi, K. C. Miao, M. Mohseni, S. Montazeri, A. Morvan, E. Mount, W. Mruczkiewicz, O. Naaman, M. Neeley, C. Neill, A. Nersisyan, H. Neven, M. Newman, J. H. Ng, A. Nguyen, M. Nguyen, M. Y. Niu, T. E. O'Brien, A. Opremcak, J. Platt, A. Petukhov, R. Potter, L. P. Pryadko, C. Quintana, P. Roushan, N. C. Rubin, N. Saei, D. Sank, K. Sankaragomathi, K. J. Satzinger, H. F. Schurkus, C. Schuster, M. J. Shearn, A. Shorter, V. Shvarts, J. Skrzynny, V. Smelyanskiy, W. C. Smith, G. Sterling, D. Strain, M. Szalay, A. Torres, G. Vidal, B. Villalonga, C. Vollgraf Heidweiller, T. White, C. Xing, Z. J. Yao, P. Yeh, J. Yoo, G. Young, A. Zalcman, Y. Zhang, and N. Zhu, "Suppressing quantum errors by scaling a surface code logical qubit," *Nature*, vol. 614, no. 7949, pp. 676–681, Feb. 2023.
- [7] A. P. M. Place, L. V. H. Rodgers, P. Mundada, B. M. Smitham, M. Fitzpatrick, Z. Leng, A. Premkumar, J. Bryon, A. Vrajitoarea, S. Sussman, G. Cheng, T. Madhavan, H. K. Babla, X. H. Le, Y. Gang, B. Jäck, A. Gyenis, N. Yao, R. J. Cava, N. P. De Leon, and A. A. Houck, "New material platform for superconducting transmon qubits with coherence times exceeding 0.3 milliseconds," *Nat. Comm.*, vol. 12, no. 1, p. 1779, Mar. 2021.
- [8] A. Y. Kitaev, "Quantum computations: algorithms and error correction," *Russ. Math. Surveys*, vol. 52, no. 6, pp. 1191–1249, Dec. 1997.
- [9] M. Amy, D. Maslov, M. Mosca, and M. Roetteler, "A Meet-in-the-Middle Algorithm for Fast Synthesis of Depth-Optimal Quantum Circuits," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 32, no. 6, pp. 818–830, Jun. 2013.
- [10] V. Kliuchnikov, D. Maslov, and M. Mosca, "Fast and efficient exact synthesis of single-qubit unitaries generated by Clifford and T gates," *Quantum Inf. Comput.*, vol. 13, no. 7&8, pp. 607–630, May 2013.
- [11] M. G. Davis, E. Smith, A. Tudor, K. Sen, I. Siddiqi, and C. Iancu, "Towards Optimal Topology Aware Quantum Circuit Synthesis," in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Denver, CO, USA: IEEE, Oct. 2020, pp. 223–234.
- [12] Z.-H. He, X.-N. Zhang, and X. Chen, "Unitary Diagonalization of the Generalized Complementary Covariance Quaternion Matrices with Application in Signal Processing," *Mathematics*, vol. 11, no. 23, p. 4840, Dec. 2023.
- [13] F. Chong, D. Franklin, and M. Martonosi, "Programming languages and compiler design for realistic quantum hardware," *Nature*, vol. 549, pp. 180–187, Sep. 2017.
- [14] C. Zhu, X. Wu, Z. Yang, J. Wang, A. Wu, S. Zheng, and X. Wang, "Quantum Compiler Design for Qubit Mapping and Routing: A Cross-Architectural Survey of Superconducting, Trapped-Ion, and Neutral Atom Systems," May 2025.
- [15] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.
- [16] A. Zulehner, A. Paler, and R. Wille, "An Efficient Methodology for Mapping Quantum Circuits to the IBM QX Architectures," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 7, pp. 1226–1236, Jul. 2019.
- [17] G. Li, Y. Ding, and Y. Xie, "Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. Providence RI USA: ACM, Apr. 2019, pp. 1001–1014.
- [18] A. A. Khan, A. Ahmad, M. Waseem, P. Liang, M. Fahmideh, T. Mikkonen, and P. Abrahamsson, "Software architecture for quantum computing systems — a systematic review," *Journal of Systems and Software*, vol. 201, p. 111682, 2023.
- [19] F. J. R. Ruiz, T. Laakkonen, J. Bausch, M. Balog, M. Barekatin, F. J. H. Heras, A. Novikov, N. Fitzpatrick, B. Romera-Paredes, J. van de Wetering, A. Fawzi, K. Meichanetzidis, and P. Kohli, "Quantum circuit optimization with AlphaTensor," *Nature Machine Intelligence*, vol. 7, p. 374–385, 2025.
- [20] Z. T. Wang, Q. Chen, Y. Du, Z. H. Yang, X. Cai, K. Huang, J. Zhang, K. Xu, J. Du, Y. Li, Y. Jiao, X. Wu, W. Liu, X. Lu, H. Xu, Y. Jin, R. Wang, H. Yu, and S. P. Zhao, "Quantum compiling with reinforcement learning on a superconducting processor," 2024.
- [21] D. Kremer, A. Javadi-Abhari, and P. Mukhopadhyay, "Optimizing the non-clifford-count in unitary synthesis using reinforcement learning," 2025.
- [22] R. Zen, J. Olle, L. Colmenarez, M. Puviani, M. Müller, and F. Marquardt, "Quantum circuit discovery for fault-tolerant logical state preparation with reinforcement learning," *Phys. Rev. X*, vol. 15, p. 041012, Oct 2025.
- [23] K. Nakaji, J. Wurtz, H. Huang, L. M. Calderón, K. Panicker, E. Kyoseva, and A. Aspuru-Guzik, "Quantum circuits as a game: A reinforcement learning agent for quantum compilation and its application to reconfigurable neutral atom arrays," Jun. 2025.
- [24] A. Kundu, "Improving thermal state preparation of sachdev–ye–kitaev model with reinforcement learning on quantum hardware," *Machine Learning: Science and Technology*, vol. 6, no. 2, p. 025066, jun 2025.
- [25] R. Romanello, D. Lizzio Bosco, J. Cossio, D. Sutulovic, G. Serra, C. Piazza, and P. Burelli, "Cnot minimal circuit synthesis: A reinforcement learning approach," in *2025 IEEE International Conference on Quantum Artificial Intelligence (QAI)*, 2025.
- [26] D. Kremer, V. Villar, H. Paik, I. Duran, I. Faro, and J. Cruz-Benito, "Practical and efficient quantum circuit synthesis and transpiling with Reinforcement Learning," May 2024.
- [27] J. Cossio, D. Lizzio Bosco, R. Romanello, G. Serra, and C. Piazza, "Alphacnot: Learning cnot minimization with model-based planning," Apr. 2026.
- [28] S. Aaronson and D. Gottesman, "Improved simulation of stabilizer circuits," *Phys. Rev. A*, vol. 70, p. 052328, Nov 2004.
- [29] D. Gottesman, "Stabilizer codes and quantum error correction," May 1997.
- [30] G. Yan, W. Wu, Y. Chen, K. Pan, X. Lu, Z. Zhou, Y. Wang, R. Wang, and J. Yan, "Quantum circuit synthesis and compilation optimization: Overview and prospects," Jun. 2024.
- [31] M. Webster, S. Koutsoumpas, and D. E. Browne, "Heuristic and optimal synthesis of cnot and clifford circuits," Mar. 2025.
- [32] K. Volanto, "Minimizing the number of two-qubit gates in clifford circuits," Master's thesis, Aalto University, Mar. 2023.
- [33] S. Bravyi, R. Shaydulin, S. Hu, and D. Maslov, "Clifford Circuit Optimization with Templates and Symbolic Pauli Gates," *Quantum*, vol. 5, p. 580, Nov. 2021.
- [34] S. Bravyi, J. A. Latone, and D. Maslov, "6-qubit optimal Clifford circuits," *npj Quantum Information*, vol. 8, no. 1, p. 79, Jul. 2022.
- [35] I. Shaik and J. van de Pol, "Cnot-optimal clifford synthesis as sat," Apr. 2025.
- [36] A. Kundu and S. Mangini, "TensorRL-QAS: Reinforcement learning with tensor networks for improved quantum architecture search," Sep. 2025.
- [37] D. Lizzio Bosco, L. Cincio, G. Serra, and M. Cerezo, "Quantum circuit pre-synthesis: Learning local edits to reduce t -count," Jan. 2026.
- [38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," Jul. 2017.
- [39] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, "LightSABRE: A Lightweight and Enhanced SABRE Algorithm," Sep. 2024.
- [40] R. Coulom, "Efficient selectivity and backup operators in monte-carlo tree search," in *Computers and Games, 5th Int. Conf., Turin, Italy, May 2006, revised papers*, 2007.
- [41] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [42] M. Doherty, M. Puviani, J. Brewer, G. Matos, D. Amaro, B. Criger, and D. T. Stephen, "Fast stabilizer state preparation via AI-optimized graph decimation," Mar. 2026.
- [43] L. Kocsis and C. Szepesvári, "Bandit based monte-carlo planning," in *Machine Learning: ECML 2006, 17th European Conference on Machine Learning*, 2006.
- [44] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. 26th Int. Conf. Mach. Learn. (ICML)*, Jun. 2009.
- [45] A. K. Prasad, V. V. Shende, I. L. Markov, J. P. Hayes, and K. N. Patel, "Data structures and algorithms for simplifying reversible circuits," *J. Emerg. Technol. Comput. Syst.*, vol. 2, no. 4, p. 277–293, Oct. 2006. [Online]. Available: <https://doi.org/10.1145/1216396.1216399>
- [46] P. Krantz, M. Kjaergaard, F. Yan, T. P. Orlando, S. Gustavsson, and W. D. Oliver, "A quantum engineer's guide to superconducting qubits," *Appl. Phys. Reviews*, vol. 6, no. 2, p. 021318, Jun. 2019.

- [47] C. M. Dawson and M. A. Nielsen, "The solovay-kitaev algorithm," *Quantum Info. Comput.*, vol. 6, no. 1, pp. 81–95, Jan. 2006.
- [48] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge: Cambridge University Press, 2000.
- [49] E. Cartan, "Sur une classe remarquable d'espaces de Riemann," *Bull. Soc. Math. Fr.*, vol. 2, pp. 214–264, 1926.
- [50] D. Wierichs, M. West, R. T. Forestano, M. Cerezo, and N. Killoran, "Recursive Cartan decompositions for unitary synthesis," Jun. 2025.
- [51] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," Jun. 2024.
- [52] R. Koenig and J. A. Smolin, "How to efficiently select an arbitrary clifford group element," *J. Math. Physics*, vol. 55, no. 12, p. 122202, Dec. 2014.
- [53] C. Gidney, "Stim: a fast stabilizer circuit simulator," *Quantum*, vol. 5, p. 497, Jul. 2021. [Online]. Available: <https://doi.org/10.22331/q-2021-07-06-497>
- [54] Qiskit, "Ai-transpiler cliffords," 2024, hugging Face model repository. Accessed: Apr. 21, 2026.
- [55] N. J. Ross and P. Selinger, "Optimal ancilla-free clifford+t approximation of z-rotations," *Quantum Inf. Comput.*, vol. 16, no. 11-12, pp. 901–953, 2016.